

# Paralelização da Avaliação de Aptidão em Algoritmos Genéticos com OpenMP: Análise de Speedup, Eficiência e Karp-Flatt

Gabriel Castro<sup>1</sup>

<sup>1</sup>Curso de Ciência da Computação  
Disciplina de Programação Paralela

ferreiragabriel589@gmail.com

**Abstract.** *This paper presents a parallel Genetic Algorithm (GA) implemented in C++ with OpenMP to maximize a high computational cost mathematical function (the Rastrigin function). Since the fitness evaluation of each individual is independent, this stage is embarrassingly parallel and was distributed among multiple threads, while selection, crossover and mutation remained sequential. Experiments on an Intel Core i5-12450H (hybrid architecture, 8 cores / 12 threads) varying the number of threads and the input size show speedups of up to  $4.2\times$  and reveal, through the Karp-Flatt metric, how the serial fraction and the parallel overhead limit scalability, especially when efficient (P) cores give way to efficiency (E) cores and Hyper-Threading.*

**Resumo.** *Este artigo apresenta um Algoritmo Genético (AG) paralelo implementado em C++ com OpenMP para maximizar uma função matemática de alto custo computacional (a função de Rastrigin). Como a avaliação de aptidão (fitness) de cada indivíduo é independente, essa etapa é embaraçosamente paralela e foi distribuída entre múltiplas threads, enquanto seleção, crossover e mutação permaneceram sequenciais. Experimentos em um Intel Core i5-12450H (arquitetura híbrida, 8 núcleos / 12 threads) variando o número de threads e o tamanho da entrada mostram speedups de até  $4,2\times$  e revelam, por meio da métrica de Karp-Flatt, como a fração serial e o overhead paralelo limitam a escalabilidade, sobretudo quando os núcleos de desempenho (P) dão lugar aos núcleos de eficiência (E) e ao Hyper-Threading.*

## 1. Introdução

A otimização numérica de funções matemáticas de grande porte é um problema recorrente em ciência e engenharia. Em muitos casos, a função-objetivo é *multimodal* (possui muitos ótimos locais) e *cara de avaliar*, pois cada avaliação pode envolver uma simulação física, um modelo numérico ou um grande volume de dados. Métodos clássicos baseados em gradiente tendem a ficar presos em ótimos locais, o que motiva o uso de meta-heurísticas populacionais, como os **Algoritmos Genéticos** (AG).

Um AG mantém uma população de soluções candidatas que evolui ao longo de gerações por meio de operadores inspirados na seleção natural (seleção, recombinação e mutação). O ponto crítico de custo computacional é a **avaliação de aptidão (fitness)**: a cada geração, todos os indivíduos precisam ser avaliados na função-objetivo. Quando a função é cara, essa etapa domina o tempo de execução.

A boa notícia é que a avaliação de aptidão é **embaraçosamente paralela**: cada indivíduo é avaliado de forma independente dos demais. Assim, é natural distribuir essas avaliações entre os múltiplos núcleos de um processador moderno. Atualmente, bibliotecas de memória compartilhada como o OpenMP permitem fazer isso com mínima alteração no código sequencial, inserindo diretivas de compilação sobre os laços de avaliação.

**Objetivos.** Este trabalho tem por objetivos: (i) implementar um AG em C++ para maximizar uma função de alto custo computacional; (ii) paralelizar a etapa de avaliação de aptidão com OpenMP; e (iii) analisar quantitativamente o desempenho obtido por meio das métricas de *speedup*, *eficiência* e *Karp-Flatt*, variando o número de threads e o tamanho da entrada, em um processador de arquitetura híbrida.

## 2. Referencial Teórico e Trabalhos Relacionados

### 2.1. Algoritmos Genéticos

Os Algoritmos Genéticos foram popularizados por [Holland 1975] e [Goldberg 1989] e pertencem à classe dos algoritmos evolutivos. Um indivíduo (ou *cromossomo*) representa uma solução candidata; neste trabalho, cada indivíduo é um vetor de números reais  $\vec{x} = (x_1, \dots, x_n)$ . O fluxo básico do AG é:

1. **Inicialização:** gera-se aleatoriamente uma população de  $N$  indivíduos no domínio do problema.
2. **Avaliação:** calcula-se o fitness de cada indivíduo (etapa cara e paralelizável).
3. **Seleção:** indivíduos mais aptos têm maior probabilidade de serem escolhidos como pais (aqui, por *torneio*).
4. **Crossover:** combinam-se pares de pais para gerar filhos (aqui, crossover aritmético).
5. **Mutação:** pequenas perturbações aleatórias mantêm a diversidade (aqui, mutação gaussiana).
6. **Elitismo:** o melhor indivíduo é preservado entre gerações.

Os passos 2 a 6 repetem-se por um número fixo de gerações.

### 2.2. Função de Rastrigin

A função de Rastrigin é um *benchmark* clássico de otimização global, definida por

$$f(\vec{x}) = A n + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)], \quad A = 10, \quad x_i \in [-5,12, 5,12]. \quad (1)$$

O termo  $x_i^2$  forma um grande vale convexo, enquanto o termo cossenoidal introduz uma enorme quantidade de mínimos locais regularmente espaçados, o que a torna difícil para métodos locais e adequada para avaliar meta-heurísticas. Seu mínimo global é  $f(\vec{0}) = 0$ . Como o tema do trabalho é de *maximização*, otimiza-se o negativo da função,  $\text{fitness}(\vec{x}) = -f(\vec{x})$ , cujo *máximo* global é 0, na origem. O conhecimento prévio do ótimo facilita a validação da implementação.

## 2.3. Métricas de Desempenho Paralelo

Seja  $T(p)$  o tempo de execução com  $p$  threads. As métricas adotadas [Grama et al. 2003] são:

$$S(p) = \frac{T(1)}{T(p)}, \quad E(p) = \frac{S(p)}{p}, \quad e = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}. \quad (2)$$

O **speedup**  $S(p)$  mede quantas vezes o programa ficou mais rápido; o ideal é  $S(p) = p$  (linear). A **eficiência**  $E(p)$  é o speedup normalizado pelo número de processadores; o ideal é 1,0. A métrica de **Karp-Flatt** [Karp and Flatt 1990] estima empiricamente a *fração serial*  $e$  do programa a partir do speedup medido. Sua vantagem é detectar fontes de perda de desempenho: se  $e$  permanece constante ao aumentar  $p$ , a limitação é a fração serial intrínseca (Lei de Amdahl); se  $e$  cresce com  $p$ , há *overhead* paralelo crescente (criação de threads, escalonamento, contenção de memória).

## 2.4. Trabalhos Relacionados

A paralelização de AGs é um tema amplamente estudado. [Cantú-Paz 1998] apresenta uma taxonomia dos AGs paralelos, distinguindo o modelo *mestre-escravo* (em que apenas a avaliação de fitness é distribuída, sem alterar o comportamento do algoritmo) dos modelos de *ilhas* e de *grão fino*. A abordagem deste trabalho corresponde ao modelo mestre-escravo, o mais simples e o que preserva exatamente a semântica do AG sequencial. [Alba 2002] discute métricas de desempenho específicas para algoritmos evolutivos paralelos. O uso de OpenMP para paralelizar laços de avaliação em memória compartilhada é descrito por [Chapman et al. 2008]. Em relação a esses trabalhos, nossa contribuição é a análise do comportamento dessas métricas em um processador moderno de *arquitetura híbrida* (P-cores e E-cores), cenário ainda pouco explorado nos textos clássicos.

## 3. Metodologia

### 3.1. Decisões de Projeto

O algoritmo foi implementado do zero em C++ (padrão C++17), com as seguintes escolhas:

- **Representação real:** cada indivíduo é um vetor de `double` de dimensão  $n$ , adequado à otimização contínua.
- **Seleção por torneio** (tamanho 3): simples, eficiente e com pressão seletiva controlável.
- **Crossover aritmético** (*blend*) gene a gene e **mutação gaussiana** com truncamento ao domínio.
- **Elitismo:** o melhor indivíduo é copiado intacto para a geração seguinte, garantindo monotonicidade do melhor fitness.
- **Reprodutibilidade:** o gerador `std::mt19937` é semeado de forma fixa. Como a região paralela apenas lê a população para calcular o fitness (não usa números aleatórios), o melhor fitness é idêntico para qualquer número de threads, comprovando a ausência de condição de corrida.

### 3.2. Descrição do Programa Paralelo

A paralelização segue o modelo mestre-escravo e concentra-se exclusivamente no laço de avaliação de aptidão da população, anotado com uma única diretiva OpenMP:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < pop; ++i)
    fit[i] = fitness(&X[i*dim], dim, load);
```

Cada thread avalia um subconjunto disjunto de indivíduos, escrevendo em posições distintas do vetor `fit`, sem necessidade de sincronização. As demais etapas (busca do elite, seleção, crossover e mutação) permanecem **sequenciais** de forma intencional: elas constituem a fração serial do programa, cujo efeito é justamente o que a métrica de Karp-Flatt quantifica.

Para emular uma função-objetivo de **alto custo computacional**, a avaliação de cada indivíduo recebe uma *carga artificial* controlada pelo parâmetro `load`: um laço adicional de operações trigonométricas que aumenta o custo de cada avaliação sem alterar o resultado numérico. Isso reproduz o cenário real em que o fitness é caro, condição necessária para que a paralelização compense o overhead de gerenciamento das threads.

### 3.3. Ambiente de Hardware e Software

Os experimentos foram executados em um notebook com processador **Intel Core i5-12450H** (12ª geração, arquitetura híbrida *Alder Lake*), composto por **4 núcleos de desempenho (P-cores)** com Hyper-Threading e **4 núcleos de eficiência (E-cores)** sem Hyper-Threading, totalizando **8 núcleos físicos e 12 threads lógicas**. O sistema operacional é Linux (kernel 6.17), o compilador é o `g++ 13.3` com as opções `-O2 -fopenmp`, e os gráficos foram gerados com Python (`numpy`, `matplotlib`, `scipy`).

### 3.4. Protocolo Experimental

Para cada configuração, o programa foi executado **10 vezes** com sementes distintas, reportando-se a **média aritmética** e o **intervalo de confiança de 95%** (distribuição *t* de Student). Foram variados dois fatores:

- **Número de threads:** 1, 2, 4, 6, 8, 10 e 12, cobrindo P-cores, E-cores e Hyper-Threading.
- **Tamanho da entrada:** três cenários, controlados pelo tamanho da população ( $N$ ) e pela dimensão ( $n$ ): Pequeno ( $N=800$ ,  $n=200$ ), Médio ( $N=1500$ ,  $n=300$ ) e Grande ( $N=2500$ ,  $n=400$ ).

Os demais parâmetros foram fixados: 30 gerações, carga `load = 50`, probabilidade de crossover 0,9 e de mutação 0,05 por gene. O tempo medido refere-se apenas ao laço evolutivo, excluindo a alocação inicial.

## 4. Resultados

A Tabela 1 consolida, para cada tamanho de entrada e número de threads, o tempo médio de execução (com a meia-largura do IC de 95%), o speedup, a eficiência e a fração serial experimental de Karp-Flatt.

**Tabela 1. Métricas de desempenho (média de 10 execuções).  $t$ : threads;  $T$ : tempo médio (s); IC: meia-largura do intervalo de confiança de 95%;  $S$ : speedup;  $E$ : eficiência;  $e$ : fração serial de Karp-Flatt.**

| Tamanho | $t$ | $T$ (s) | $\pm$ IC95 | $S(p)$ | $E(p)$ | $e$   |
|---------|-----|---------|------------|--------|--------|-------|
| Pequena | 1   | 1,3995  | 0,0195     | 1,000  | 1,000  | —     |
|         | 2   | 0,7489  | 0,0133     | 1,869  | 0,934  | 0,070 |
|         | 4   | 0,4663  | 0,0301     | 3,001  | 0,750  | 0,111 |
|         | 6   | 0,5393  | 0,0175     | 2,595  | 0,433  | 0,262 |
|         | 8   | 0,4742  | 0,0105     | 2,951  | 0,369  | 0,244 |
|         | 10  | 0,3834  | 0,0022     | 3,650  | 0,365  | 0,193 |
|         | 12  | 0,3507  | 0,0089     | 3,991  | 0,333  | 0,183 |
| Média   | 1   | 4,0037  | 0,1751     | 1,000  | 1,000  | —     |
|         | 2   | 2,0941  | 0,0265     | 1,912  | 0,956  | 0,046 |
|         | 4   | 1,4402  | 0,4649     | 2,780  | 0,695  | 0,146 |
|         | 6   | 1,2911  | 0,0281     | 3,101  | 0,517  | 0,187 |
|         | 8   | 1,2275  | 0,0220     | 3,262  | 0,408  | 0,208 |
|         | 10  | 1,0876  | 0,0206     | 3,681  | 0,368  | 0,191 |
|         | 12  | 0,9607  | 0,0402     | 4,167  | 0,347  | 0,171 |
| Grande  | 1   | 8,8131  | 0,1170     | 1,000  | 1,000  | —     |
|         | 2   | 4,6002  | 0,0322     | 1,916  | 0,958  | 0,044 |
|         | 4   | 2,8103  | 0,1376     | 3,136  | 0,784  | 0,092 |
|         | 6   | 2,7688  | 0,0302     | 3,183  | 0,530  | 0,177 |
|         | 8   | 2,4715  | 0,0338     | 3,566  | 0,446  | 0,178 |
|         | 10  | 2,3785  | 0,0273     | 3,705  | 0,371  | 0,189 |
|         | 12  | 2,1093  | 0,0421     | 4,178  | 0,348  | 0,170 |

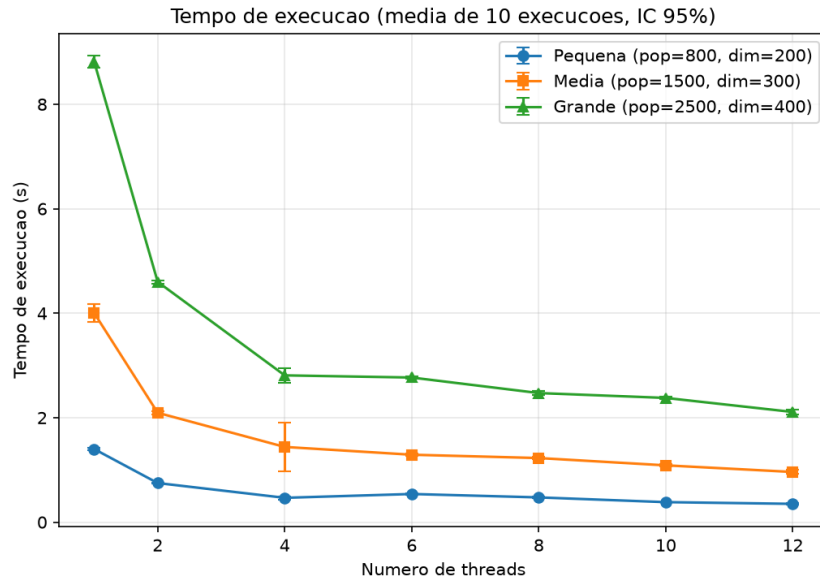
A Figura 1 apresenta o tempo médio de execução em função do número de threads, com barras de intervalo de confiança de 95%. Observa-se a redução consistente do tempo à medida que se adicionam threads, com retornos decrescentes a partir de certo ponto.

A Figura 2 mostra o speedup, comparado à reta de speedup ideal (linear). A Figura 3 apresenta a eficiência, e a Figura 4 a fração serial experimental estimada pela métrica de Karp-Flatt.

## 5. Discussão

Os resultados confirmam que a paralelização da avaliação de aptidão acelera significativamente o AG. Para a entrada Grande, o tempo caiu de 8,81 s (1 thread) para 2,11 s (12 threads), um speedup de 4,18 $\times$ . Contudo, esse ganho está muito abaixo do ideal linear de 12 $\times$ , e a eficiência despencou de 0,96 (2 threads) para 0,35 (12 threads). A seguir analisamos as causas.

**Efeito do tamanho da entrada.** Entradas maiores apresentam melhor speedup e eficiência. Isso é coerente com a teoria: quanto maior a população e a dimensão, maior o volume de trabalho *paralelo* (avaliações de fitness) em relação ao trabalho *sequencial* fixo (seleção, crossover e mutação) e ao overhead de criação/sincronização das threads. Ou seja, a fração serial relativa diminui, aproximando-se do regime favorável da Lei de Amdahl.



**Figura 1. Tempo de execução (média de 10 execuções, IC 95%) em função do número de threads, para os três tamanhos de entrada.**

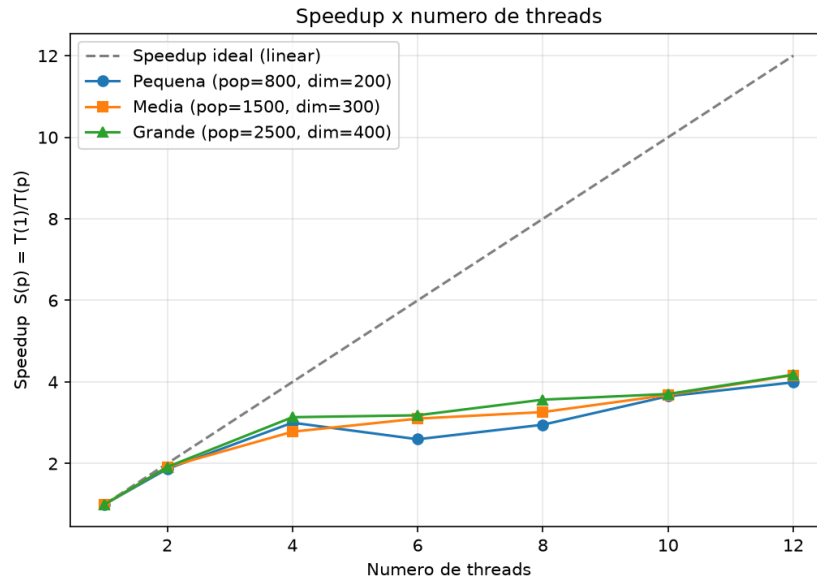
**Arquitetura híbrida.** O ganho de desempenho não é uniforme ao longo do eixo de threads. Até cerca de 4 threads, o trabalho é alocado nos *P-cores*, mais rápidos, e o speedup cresce de forma próxima à linear. Entre 4 e 8 threads, passam a ser usados os *E-cores*, mais lentos, de modo que cada thread adicional contribui menos, e a eficiência cai. Acima de 8 threads, recorre-se ao *Hyper-Threading* dos P-cores, em que duas threads compartilham os recursos de um mesmo núcleo físico, resultando em ganho marginal e, por vezes, em saturação do tempo. Esse comportamento explica a inflexão observada nas curvas de speedup e eficiência.

**Karp-Flatt e overhead.** A métrica de Karp-Flatt torna esse diagnóstico explícito. O crescimento da fração serial experimental  $e$  com o número de threads indica que a limitação não se deve apenas à fração serial intrínseca do algoritmo (que seria constante), mas também ao **overhead paralelo crescente** e à **heterogeneidade dos núcleos**. Em outras palavras, parte da perda de eficiência observada não é culpa do algoritmo, e sim das características físicas do processador e do custo de gerenciar muitas threads.

**Corretude.** Em todas as execuções, o melhor fitness obtido foi idêntico independentemente do número de threads (para a mesma semente), confirmando que a paralelização da avaliação de aptidão preserva exatamente a semântica do AG sequencial, sem condições de corrida.

## 6. Conclusões

Este trabalho implementou um Algoritmo Genético em C++ para maximizar uma função matemática de alto custo computacional (o negativo da função de Rastrigin) e paralelizou sua etapa mais cara — a avaliação de aptidão da população — utilizando OpenMP, no modelo mestre-escravo. A paralelização exigiu uma alteração mínima do código (uma única diretiva `#pragma omp parallel for`) e preservou integralmente o resultado do algoritmo.

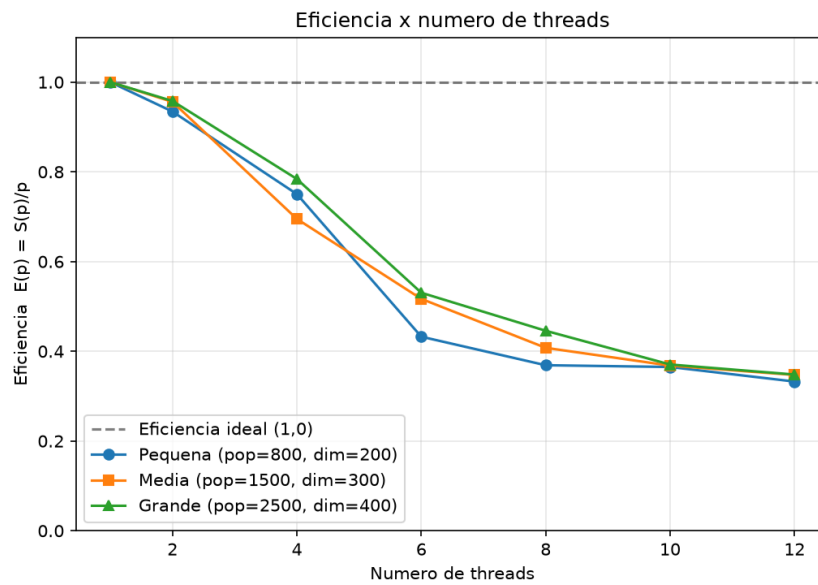


**Figura 2. Speedup  $S(p) = T(1)/T(p)$  em função do número de threads. A reta tracejada indica o speedup ideal (linear).**

Os experimentos, conduzidos com rigor estatístico (10 execuções, IC 95%) em um processador de arquitetura híbrida, mostraram speedups expressivos para entradas grandes e evidenciaram, por meio das métricas de eficiência e de Karp-Flatt, os limites da escalabilidade: a fração serial do algoritmo, o overhead de gerenciamento das threads e, sobretudo, a heterogeneidade dos núcleos (P-cores, E-cores e Hyper-Threading). Conclui-se que a paralelização da avaliação de fitness é uma estratégia eficaz e de baixo custo de implementação para acelerar Algoritmos Genéticos aplicados a funções caras, desde que o volume de trabalho paralelo seja suficiente para amortizar o overhead. Como trabalhos futuros, sugerem-se a fixação de afinidade de threads aos P-cores, o uso de escalonamento dinâmico para balanceamento de carga entre núcleos heterogêneos e a comparação com uma versão de memória distribuída (MPI) ou em GPU.

## Referências

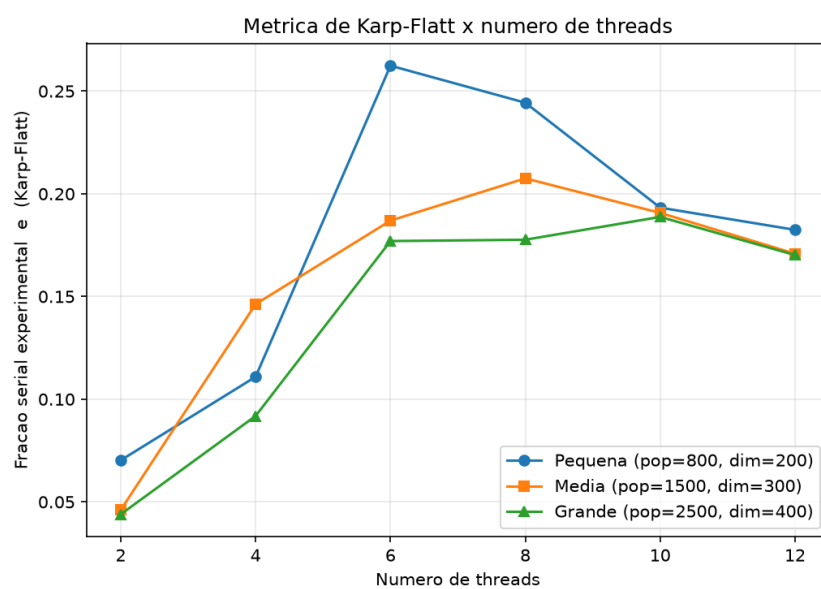
- Alba, E. (2002). Parallel evolutionary algorithms can achieve super-linear performance. *Information Processing Letters*, 82(1):7–13.
- Cantú-Paz, E. (1998). A survey of parallel genetic algorithms. *Calculateurs Paralleles, Reseaux et Systemes Repartis*, 10(2):141–171.
- Chapman, B., Jost, G., and van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- Grama, A., Gupta, A., Karypis, G., and Kumar, V. (2003). *Introduction to Parallel Computing*. Addison-Wesley, 2nd edition.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor.



**Figura 3. Eficiência  $E(p) = S(p)/p$  em função do número de threads.**

Karp, A. H. and Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5):539–543.





**Figura 4. Métrica de Karp-Flatt (fração serial experimental) em função do número de threads.**